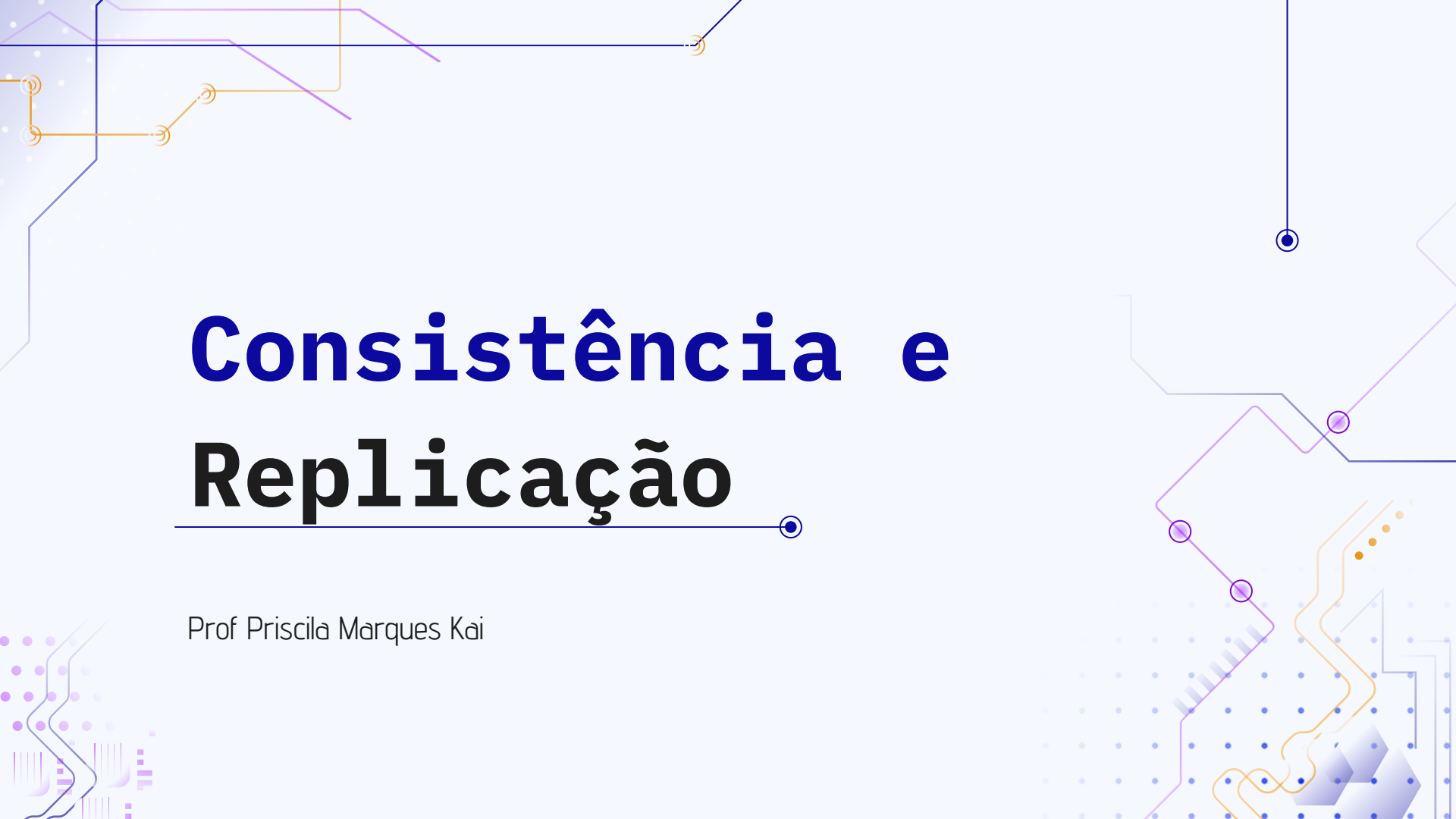




Consistência e Replicação

Prof Priscila Marques Kai

CONSISTÊNCIA E REPLICAÇÃO

Para que serve a **replicação** em sistemas distribuídos?

CONSISTÊNCIA E REPLICAÇÃO

Para que serve a **replicação** em sistemas distribuídos?

A replicação de dados aumenta a **confiabilidade** ou melhora o **desempenho** de um sistema distribuído, de modo a servir como uma técnica para alcançar a **escalabilidade**.

CONSISTÊNCIA E REPLICAÇÃO

Para que serve a **replicação** em sistemas distribuídos?

A replicação de dados aumenta a **confiabilidade** ou melhora o **desempenho** de um sistema distribuído, de modo a servir como uma técnica para alcançar a **escalabilidade**.

- **Confiabilidade**: em um sistema de arquivos replicado, caso uma das réplicas falhe ou corrompa, é possível alternar para uma das réplicas.

CONSISTÊNCIA E REPLICAÇÃO

Para que serve a **replicação** em sistemas distribuídos?

A replicação de dados aumenta a **confiabilidade** ou melhora o **desempenho** de um sistema distribuído, de modo a servir como uma técnica para alcançar a **escalabilidade**.

- **Confiabilidade**: em um sistema de arquivos replicado, caso uma das réplicas falhe ou corrompa, é possível alternar para uma das réplicas.
- **Desempenho**: dados replicados possibilitando a divisão da carga de trabalho entre os processos que acessam os dados.

CONSISTÊNCIA E REPLICAÇÃO

Para que serve a **replicação** em sistemas distribuídos?

A replicação de dados aumenta a **confiabilidade** ou melhora o **desempenho** de um sistema distribuído, de modo a servir como uma técnica para alcançar a **escalabilidade**.

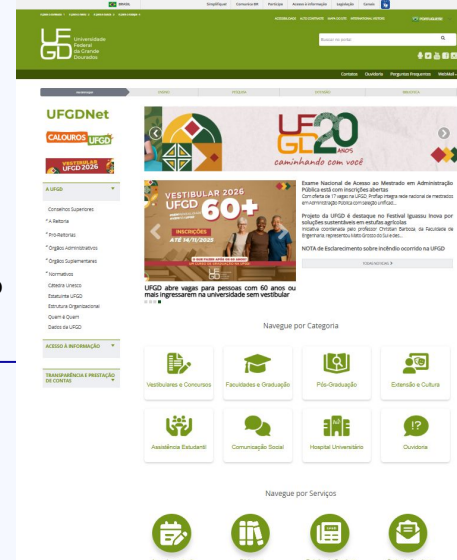
- **Confiabilidade**: em um sistema de arquivos replicado, caso uma das réplicas falhe ou corrompa, é possível alternar para uma das réplicas.
- **Desempenho**: dados replicados possibilitando a divisão da carga de trabalho entre os processos que acessam os dados.
- **Escalabilidade**: posicionar cópia dos dados mais próximo do processo que os utiliza, reduz o tempo de acesso aos dados.

CONSISTÊNCIA E REPLICAÇÃO

Para que serve a **replicação** em sistemas distribuídos?

Exemplo: Tempo de acesso a uma página da web.

- O carregamento de uma página de um servidor web remoto pode, às vezes, levar até segundos para ser concluído.
- Para melhorar o desempenho
 - os navegadores armazenam localmente uma cópia de uma página da web acessada anteriormente (em cache).
 - o tempo de acesso percebido pelo usuário é reduzido.



CONSISTÊNCIA E REPLICAÇÃO

Qual a relação entre **replicação** e **consistência** em sistemas distribuídos?

Ter várias cópias pode levar a **problemas de consistência**.

- A modificação de uma cópia a torna diferente das demais.
 - Necessidade de realizar modificações em todas as cópias para garantir a consistência.
- Quando e como as modificações são feitas determinam o custo da replicação.

CONSISTÊNCIA E REPLICAÇÃO

Qual a relação entre **replicação** e **consistência** em sistemas distribuídos?

Exemplo: Tempo de acesso a uma página da web.

Considerando o cenário do exemplo anterior, onde o navegador guarda uma cópia de uma página da web, podemos ter alguns problemas em relação à cópias:

- Na **atualização da página**: se a página tiver sido modificada posteriormente, as modificações não terão sido propagadas para as cópias, tornando-as **desatualizadas**.
- **Solução**: não permitir que o navegador mantenha cópias locais, deixando o servidor totalmente responsável pela replicação (pode resultar em tempos ruins de acesso).

CONSISTÊNCIA E REPLICAÇÃO

Qual a relação entre **replicação** e **consistência** em sistemas distribuídos?

Exemplo: Tempo de acesso a uma página da web.

Considerando o cenário do exemplo anterior, onde o navegador guarda uma cópia de uma página da web, podemos ter alguns problemas em relação à cópias:

- Na **atualização da página**: se a página tiver sido modificada posteriormente, as modificações não terão sido propagadas para as cópias, tornando-as **desatualizadas**.
- **Solução 2**: permitir que o servidor web invalide ou atualize cada cópia em cache (exige acompanhamento do servidor, **afetando o desempenho geral do servidor**).

CONSISTÊNCIA E REPLICAÇÃO

Qual a relação entre **replicação** e **consistência** em sistemas distribuídos?

Devido a isso, é necessário analisar **desempenho** versus **escalabilidade** em sistemas distribuídos.

CONSISTÊNCIA E REPLICAÇÃO

Replicação como técnica de **escalabilidade**

Problemas de escalabilidade geralmente se manifestam na forma de **problemas de desempenho**.

- Cópias dos dados próximas aos processos que os utilizam pode:
 - melhorar o desempenho e
 - reduzir o tempo de acesso.
- No entanto, **manter as cópias atualizadas** pode:
 - exigir **maior largura de banda da rede**

CONSISTÊNCIA E REPLICAÇÃO

Replicação como técnica de **escalabilidade**

Exemplo: Imagine um sistema que mantém várias réplicas de um mesmo dado.

- De tempos em tempos, o sistema atualiza cada réplica, substituindo completamente a versão anterior pela nova.

CONSISTÊNCIA E REPLICAÇÃO

Replicação como técnica de **escalabilidade**

Exemplo: Imagine um sistema que mantém várias réplicas de um mesmo dado.

- De tempos em tempos, o sistema atualiza cada réplica, substituindo completamente a versão anterior pela nova.

Agora, suponha que:

- Um processo P acessa uma réplica local **N** vezes por segundo;
- Essa mesma réplica seja atualizada **M** vezes por segundo.

CONSISTÊNCIA E REPLICAÇÃO

Replicação como técnica de **escalabilidade**

Exemplo: Imagine um sistema que mantém várias réplicas de um mesmo dado.

- Processo P: **N** vezes por segundo;
- Atualização da réplica: **M** vezes por segundo.

Se **N** < **M**: muitas atualizações para poucos acessos:

- redução na carga da rede e melhora o desempenho.
- muitas versões atualizadas da réplica local nunca serão acessadas por P (comunicação de rede desnecessária para essas versões);
- maior a chance de inconsistência temporária entre as réplicas;
- sistemas de **consistência forte**

CONSISTÊNCIA E REPLICAÇÃO

Replicação como técnica de **escalabilidade**

Exemplo: Imagine um sistema que mantém várias réplicas de um mesmo dado.

- Processo P: **N** vezes por segundo;
- Atualização da réplica: **M** vezes por segundo.

Se **N** > **M**: muitas acessos para poucas atualizações:

- redução na carga da rede e melhora o desempenho;
- pode haver atraso na propagação das mudanças (réplica com dados desatualizados).
- sistemas onde a **consistência eventual** é suficiente

CONSISTÊNCIA E REPLICAÇÃO

Consistência em Sistemas com Múltiplas Réplicas

Garantir que todas as cópias estejam sempre iguais pode causar problemas de escalabilidade.

- Necessidade de que todas as réplicas concordem sobre a ordem e o momento exato em que cada atualização deve ocorrer (**acordo**).
 - pode envolver o uso de **timestamps**; ou um **coordenador** para decidir a sequência das atualizações.
 - **exige muitas trocas de mensagens** entre os nós (consumo de tempo e largura de banda).

CONSISTÊNCIA E REPLICAÇÃO



Consistência em Sistemas com Múltiplas Réplicas

Problema: manter todas as cópias consistentes exige sincronização global, que é custosa e lenta.

- **Consistência perfeita**: influencia negativamente no desempenho

Solução:

- **Consistência eventual**: permite pequenas diferenças temporárias entre as réplicas.
 - com o tempo, todas as réplicas acabam convergindo para o mesmo estado.

CONSISTÊNCIA E REPLICAÇÃO

Consistência em Sistemas com Múltiplas Réplicas

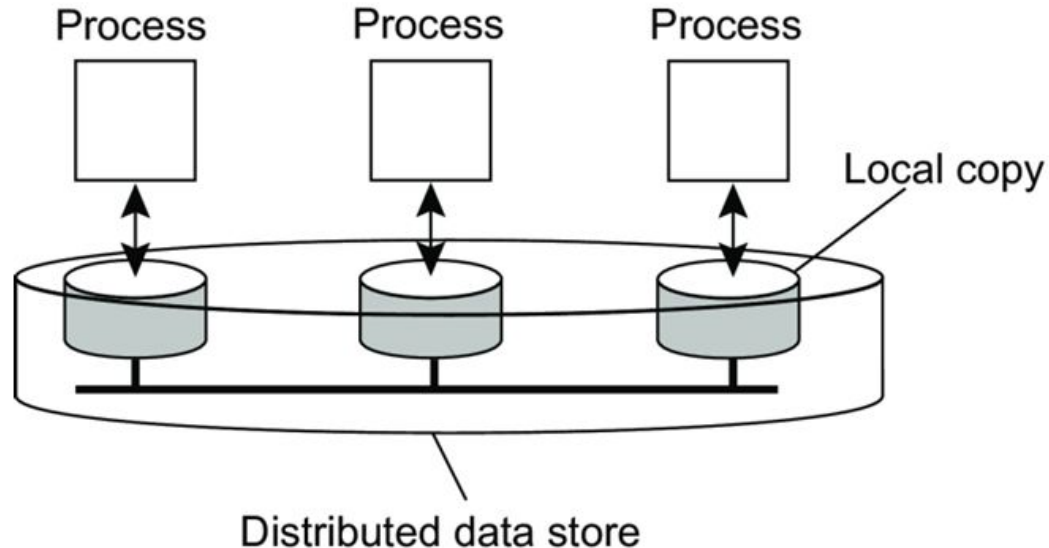
O armazenamento de dados pode ser fisicamente distribuído em várias máquinas.

- acesso a dados por processos através de uma cópia local (ou próxima);
- operações de escrita são propagadas para as outras cópias
 - Ao alterar dados tem-se uma operação de dados, caso contrário, é classificada como uma operação de leitura.

CONSISTÊNCIA E REPLICAÇÃO

Consistência em Sistemas com Múltiplas Réplicas

O armazenamento de dados pode ser fisicamente distribuído em várias máquinas.



CONSISTÊNCIA E REPLICAÇÃO

Modelos de **consistência** **centrados em dados**

Modelo que funciona como um contrato entre os processos e o armazenamento de dados.

- Dada uma operação de leitura por um processo, espera-se que a operação retorne um valor atualizado sobre o dado.

Mas como definir qual a **última operação** sem relógio global?

CONSISTÊNCIA E REPLICAÇÃO

Modelos de **consistência** **centrados em dados**

Mas como definir qual a **última operação** sem relógio global?

- Necessário escolher um **modelo de consistência**:
 - cada modelo possui restrições sobre os valores que uma operação de leitura em um item de dados pode retornar;

CONSISTÊNCIA E REPLICAÇÃO

Ordenação consistente de operações

Uma parte importante dos modelos de consistência vem da área de **programação paralela**.

- compartilhamento de recursos entre vários processos, com **acesso simultâneo**;
- modelos precisavam descrever como operações simultâneas deviam acontecer quando os dados são **compartilhados e replicados** em vários nós do sistema;
- objetivo manter uma **ordem coerente nas operações** (consistência dos dados).

CONSISTÊNCIA E REPLICAÇÃO

Ordenação consistente de operações

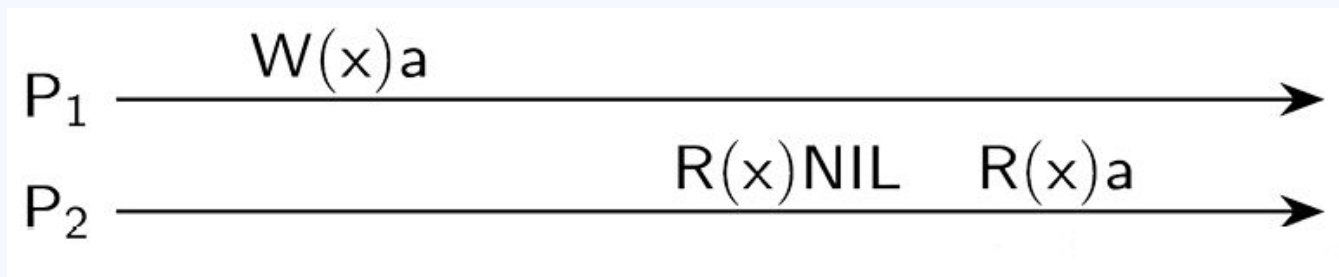
Consistência Sequencial

Para entender o modelo de consistência sequencial, considere que cada processo ($P_1, P_2, \dots$) possua uma linha do tempo, e nela conste as operações que ele executa sobre os dados.

CONSISTÊNCIA E REPLICAÇÃO

Ordenação consistente de operações

Consistência Sequencial



$W_i(x)a$ → escrita pelo processo P_i no item de dado x , atribuindo o valor a .

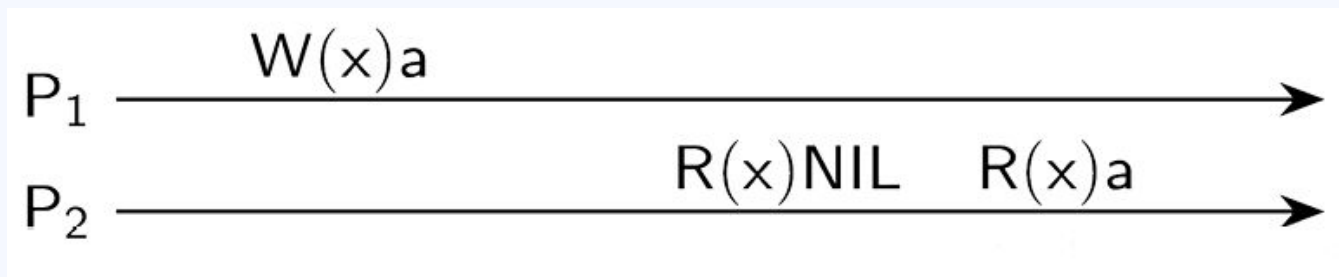
$R_i(x)b$ → leitura pelo processo P_i do item x e obtém o valor b .

Cada item de dado começa com o valor **NIL** .

CONSISTÊNCIA E REPLICAÇÃO

Ordenação consistente de operações

Consistência Sequencial



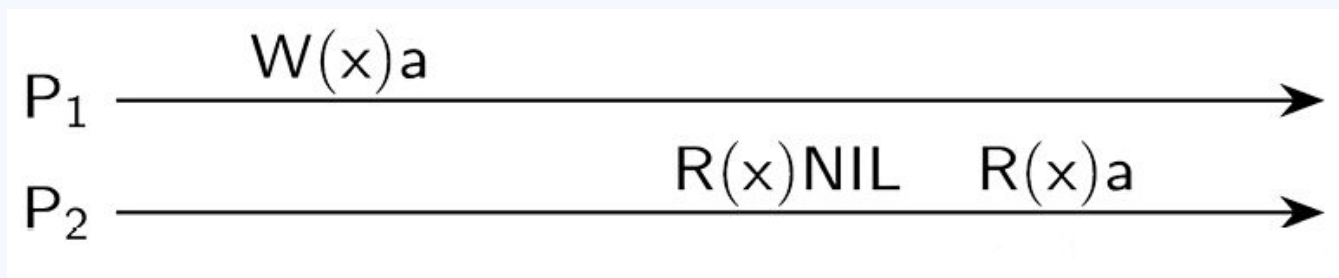
Na linha de P_1 :

- P_1 executa a operação $W_1(x)a$
- gravação ocorre na **cópia local** de P_1 e depois é **propagada** para as réplicas do dado no sistema.

CONSISTÊNCIA E REPLICAÇÃO

Ordenação consistente de operações

Consistência Sequencial



Na linha de P_2 :

- P_2 lê x antes da atualização chegar até ele
 - P_2 lê o valor **NIL** (valor inicial);
 - quando a atualização se propaga, P_2 lê o valor **a**.

CONSISTÊNCIA E REPLICAÇÃO

Ordenação consistente de operações

Consistência Sequencial

Um sistema de dados é dito **sequencialmente consistente** quando:

- *O resultado de qualquer execução é equivalente ao que seria obtido se todas as operações de leitura e escrita de todos os processos fossem executadas em uma ordem sequencial, respeitando a ordem das operações de cada processo individualmente, conforme especificado em seu respectivo programa.*

CONSISTÊNCIA E REPLICAÇÃO

Ordenação consistente de operações

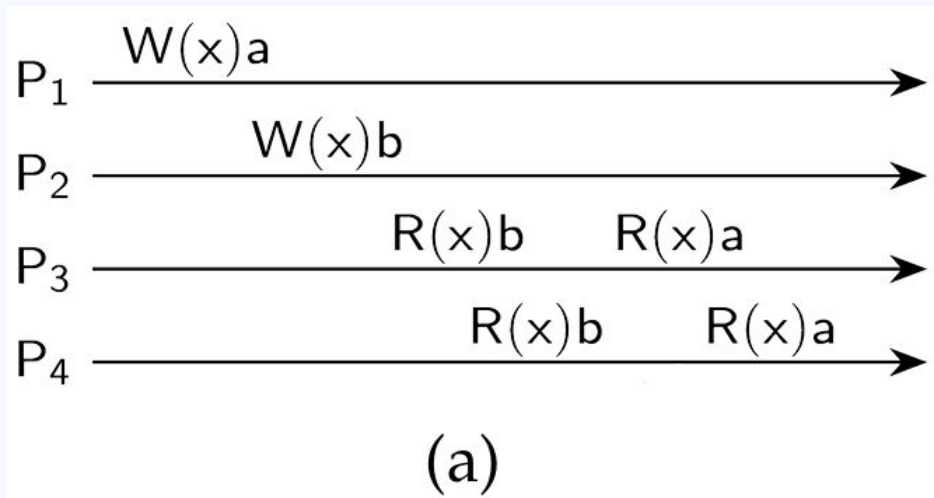
Consistência Sequencial

Quando vários processos executam concorrentemente, qualquer **interleaving válido** (intercalação entre operações de leitura e escrita) é um comportamento aceitável, desde que todos os processos enxerguem o mesmo interleaving.

CONSISTÊNCIA E REPLICAÇÃO

Ordenação consistente de operações

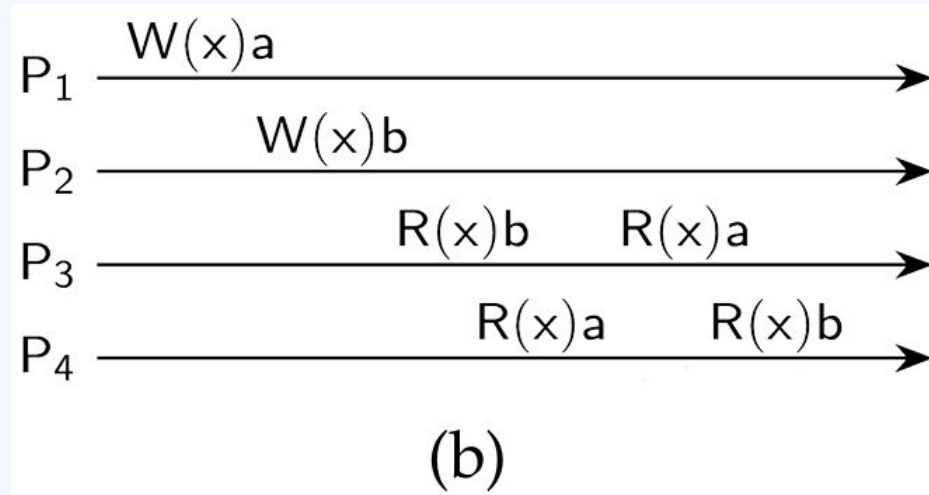
Consistência Sequencial



CONSISTÊNCIA E REPLICAÇÃO

Ordenação consistente de operações

Consistência Sequencial



CONSISTÊNCIA E REPLICAÇÃO

Ordenação consistente de operações

Consistência Sequencial

Considere processos executando concorrentemente, chamados P1, P2 e P3.

Dados manipulados: **x**, **y** e **z**, armazenados em um repositório de dados compartilhado.

Process P ₁	Process P ₂	Process P ₃
$x \leftarrow 1;$ $\text{print}(y, z);$	$y \leftarrow 1;$ $\text{print}(x, z);$	$z \leftarrow 1;$ $\text{print}(x, y);$

CONSISTÊNCIA E REPLICAÇÃO

Ordenação consistente de operações

Consistência Sequencial

- Cada variável pode assumir apenas dois valores possíveis (**0- inicial ou 1**)
- Cada processo imprime um **par de bits**. As saídas formarão uma **sequência de 6 bits** que caracteriza uma execução específica do sistema.

Process P ₁	Process P ₂	Process P ₃
x ← 1; print(y,z);	y ← 1; print(x,z);	z ← 1; print(x,y);

CONSISTÊNCIA E REPLICAÇÃO

Ordenação consistente de operações

Consistência Sequencial

Execution 1	Execution 2	Execution 3	Execution 4
P ₁ : x ← 1; P ₁ : print(y,z); P ₂ : y ← 1; P ₂ : print(x,z); P ₃ : z ← 1; P ₃ : print(x,y);	P ₁ : x ← 1; P ₂ : y ← 1; P ₂ : print(x,z); P ₁ : print(y,z); P ₃ : z ← 1; P ₃ : print(x,y);	P ₂ : y ← 1; P ₃ : z ← 1; P ₃ : print(x,y); P ₂ : print(x,z); P ₁ : x ← 1; P ₁ : print(y,z);	P ₂ : y ← 1; P ₁ : x ← 1; P ₃ : z ← 1; P ₂ : print(x,z); P ₁ : print(y,z); P ₃ : print(x,y);
<i>Prints:</i> 001011 <i>Signature:</i> 00 10 11	<i>Prints:</i> 101011 <i>Signature:</i> 10 10 11	<i>Prints:</i> 010111 <i>Signature:</i> 11 01 01	<i>Prints:</i> 111111 <i>Signature:</i> 11 11 11
(a)	(b)	(c)	(d)

CONSISTÊNCIA E REPLICAÇÃO

Ordenação consistente de operações

Consistência Sequencial

Exemplos de assinaturas impossíveis: **000000,001001**